

Pipeline propuesto para la aplicacion de extraccion de PDFs

Guion tecnico y funcional para reunion

Objetivo general

La aplicacion debe recibir un documento de entrada, que puede ser un PDF con texto embebido, un PDF escaneado o una imagen. A partir de ese documento, el sistema debe identificar de que cliente y producto/familia se trata, extraer la informacion relevante y devolver un JSON estructurado conforme al contrato definido para ese cliente/producto.

El objetivo no es programar un extractor rigido por cada version de formulario, sino construir un pipeline flexible basado en contrato de campos, OCR, texto embebido, validacion y apoyo de un modelo LLM para interpretar formularios aunque cambie ligeramente el layout.

1. Entrada del documento

La aplicacion recibira un archivo desde una API, inicialmente FastAPI en Python. El cliente C# enviara el PDF o imagen mediante una peticion HTTP.

El endpoint recibira:

- archivo: PDF o imagen.
- cliente opcional: si ya se conoce.
- producto opcional: si ya se conoce.
- modo debug opcional.

Si el cliente o producto no vienen informados, el sistema intentara detectarlos automaticamente.

2. Normalizacion inicial

El primer paso sera normalizar el documento para trabajar siempre con una estructura comun.

Si es PDF:

- se leera el texto embebido con PyMuPDF u otra libreria similar;
- se renderizaran paginas a imagen si hace falta OCR;
- se conservara informacion por pagina.

Si es imagen:

- se tratara como una pagina renderizada;
- se enviara directamente al OCR.

El resultado de esta fase sera una estructura interna parecida a:

```
{
  "pages": [
    {
      "page_number": 1,
      "embedded_text": "...",
      "ocr_text": "...",
      "blocks": [],
      "image": "..."
    }
  ]
}
```

3. Extraccion base: texto, OCR y posiciones

El sistema no debe depender de una unica fuente.

Debe intentar obtener:

- texto embebido del PDF;
- texto OCR visual;
- bloques con coordenadas;
- posibles checks o marcas visuales;
- estructura aproximada por paginas y secciones.

Esto permite comparar fuentes. Por ejemplo, si el texto embebido dice una cosa pero el OCR detecta otra cerca de un checkbox, el sistema puede marcar una discrepancia o pedir resolucion semantica.

4. Identificacion del cliente/producto

Con el texto extraido, el sistema debe identificar el perfil aplicable.

Ejemplo:

```
{
  "cliente": "Raloe",
  "producto": "CRONO",
  "confidence": 0.94
}
```

Esta deteccion puede usar:

- palabras clave;
- cabeceras;
- referencias de producto;
- nombres comerciales;
- estructura del documento;
- eventualmente un LLM si hay ambigüedad.

Si no se detecta ningun perfil conocido, la respuesta no debe romper. Debe devolver un estado del tipo:

```
{
  "status": "not_supported",
  "message": "No se reconoce el cliente/producto"
}
```

5. Carga del contrato desde base de datos

Una vez identificado el perfil, el sistema consultara en base de datos el contrato activo para ese cliente/producto.

El contrato define que debe devolver el JSON final.

Ejemplo:

```
{
  "cliente": "Raloe",
  "producto": "CRONO",
  "secciones": [
    "general",
    "Caracteristicas",
    "Traccion_electrica",
    "Opciones"
  ],
  "campos": [
    {
      "nombre": "Velocidad",
      "seccion": "Caracteristicas",
      "tipo": "texto",
      "alias": ["Velocidad", "Vel."],
      "valor_defecto": ""
    },
    {
      "nombre": "Mono",
      "seccion": "Caracteristicas",
      "tipo": "si_no",
      "alias": ["Mono"],
      "valor_defecto": ""
    }
  ]
}
```

El usuario podra mantener este contrato desde un pequeno front:

- anadir campos;
- quitar campos;
- activar o desactivar campos;
- anadir alias;
- cambiar seccion;
- indicar tipo de dato;
- convertir campos extra en campos oficiales.

El usuario no deberia tener que configurar coordenadas, radios, offsets ni detalles tecnicos.

6. Generacion de evidencias candidatas

Antes de pedir el JSON final, el sistema genera evidencias.

Ejemplo:

```
{
  "campo": "Velocidad",
  "candidatos": [
    {
      "valor": "1,00 m/s",
      "fuente": "embedded_text",
      "pagina": 1
    },
    {
      "valor": "1,00 m/s",
      "fuente": "ocr",
      "pagina": 1
    }
  ]
}
```

Para checks, puede generar candidatos como:

```
{
  "campo": "Tri",
  "candidatos": [
    {
      "valor": "Si",
      "fuente": "checkbox_visual",
      "pagina": 1
    }
  ]
}
```

Esta fase puede combinar reglas deterministas simples:

- buscar etiquetas;
- buscar alias;
- mirar texto cercano;
- detectar checks;
- detectar secciones;
- detectar campos no esperados.

7. Resolucion con LLM

El LLM no debería recibir solo una imagen o un PDF sin control. Debería recibir un prompt estructurado con:

- contrato esperado;
- texto embebido;
- OCR;
- evidencias candidatas;
- reglas de salida;
- ejemplos si existen;
- obligacion de devolver JSON valido.

La instruccion principal seria:

Devuelve unicamente los campos definidos en el contrato. Si un campo no aparece, devuelve "". Si detectas campos no contemplados, anadelos a Campos_extra. No inventes valores. Respeta exactamente los nombres de campo.

El LLM actúa como resolver semántico, especialmente útil cuando:

- el layout cambia;
- una etiqueta cambia ligeramente;
- un campo se mueve;
- hay valores sin etiqueta clara;
- hay discrepancia entre OCR y texto embebido;
- aparecen campos nuevos.

8. Validación del JSON

La respuesta del LLM nunca debe aceptarse sin validar.

El sistema debe comprobar:

- que el JSON es válido;
- que contiene todas las secciones activas;
- que contiene todos los campos activos;
- que no faltan claves obligatorias;
- que los nombres son compatibles con C#;
- que los tipos de datos son correctos;
- que los valores Si/No son coherentes;
- que los campos desconocidos van a Campos_extra.

Si falta algo, se rellena con "".

Si sobra algo, se mueve a Campos_extra o se descarta según política.

9. Campos extra

El JSON final debe incluir siempre una sección global:

```
"Campos_extra": {}
```

Si el sistema detecta campos no definidos en contrato, los añade ahí:

```
"Campos_extra": {
  "Micros_de_Freno": "MECANICO",
  "Central_en_foso": "No"
}
```

Esto permite detectar cambios del formulario sin romper la integración.

En el front, el usuario podrá revisar estos campos extra y decidir:

- Añadir al contrato.
- Ignorar.
- Mantener como extra.

10. Respuesta final a C#

La API devolverá un JSON estable, por ejemplo:

```
{
  "data": {
    "general": {},
    "Caracteristicas": {},
    "Opciones": {},
    "Campos_extra": {}
  },
  "metadata": {
    "cliente": "Raloe",
    "producto": "CRONO",
    "status": "ok",
    "confidence": 0.92,
    "warnings": []
  }
}
```

Estados posibles:

- ok
- partial
- not_supported
- validation_error

La premisa importante es que la aplicacion no debe romper si falta una seccion o campo. Si el contrato espera un campo y no se encuentra, se devuelve vacio.

11. Modo diagnostico

Para desarrollo y soporte, conviene tener un modo debug.

Este modo devolveria:

- paginas detectadas;
- texto OCR;
- texto embebido;
- campos candidatos;
- campos no encontrados;
- discrepancias;
- campos extra;
- confianza por campo.

Esto no seria la respuesta normal para C#, sino una herramienta interna para mejorar el sistema.

12. Evolucion esperada

Fase inicial:

- extractor determinista para Raloe-CRONO actual;
- contrato JSON;
- FastAPI;
- tests contra PDFs conocidos.

Fase intermedia:

- contrato en base de datos;
- front para gestionar campos;

- deteccion de campos extra;
- endpoint debug;
- comparacion texto embebido vs OCR.

Fase avanzada:

- integracion LLM local o HuggingFace;
- prompt por cliente/producto;
- ejemplos validados;
- resolucion semantica;
- revision asistida de discrepancias.

Fase final:

- el usuario sube un PDF;
- el sistema detecta cliente/producto;
- consulta contrato activo;
- extrae texto/OCR;
- el LLM resuelve el JSON;
- se valida la salida;
- se devuelven datos a C#;
- los cambios nuevos aparecen en Campos_extra;
- el usuario puede actualizar el contrato desde el front sin tocar codigo.